

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1-CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1-Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Ashika.B	Reg.No.:	192521336
Department:		Date:	

ANNEXURE A Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s

- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct.

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANNEXURE A

Below are **exam-ready answers** for all 5 questions. I've written them in a clear **constraint → design → working → justification** format.

ANNEXURE A – Constraint-Based Problem Solving

1. I/O Communication Mechanism for a Real-Time Embedded System

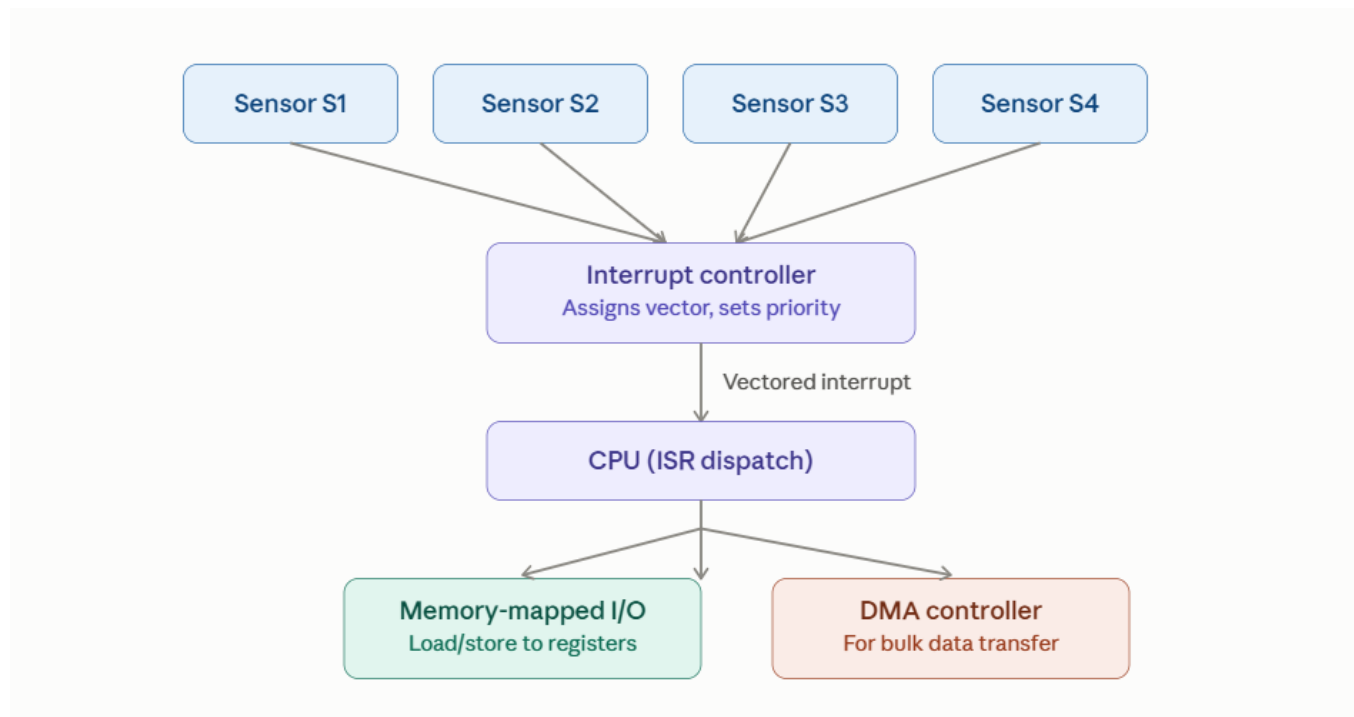
Constraints

- CPU utilization must be **below 15%**.
- Multiple sensors generate interrupts every **1 ms**.
- Use **vectored interrupt handling**.
- Select between **memory-mapped I/O** and **I/O-mapped I/O** for optimal latency.

Solution

A **vectored interrupt-based memory-mapped I/O mechanism** is suitable for the real-time embedded system. Each sensor is connected to an interrupt controller, and every sensor is assigned a unique interrupt vector.

Architecture



Working

1. Sensors continuously monitor physical parameters.
2. When a sensor requires CPU attention, it generates an interrupt.

- 3. The interrupt controller identifies the source.
- 4. The corresponding **interrupt vector** directly points to the appropriate ISR.
- 5. The CPU executes the ISR and reads the sensor register.
- 6. After servicing, the CPU returns to the previous task.
- 7. **DMA** can be used for larger sensor data transfers to further reduce CPU utilization.

Memory-Mapped vs I/O-Mapped I/O

Feature	Memory-Mapped I/O		I/O-Mapped I/O	
Address space	Shared	with memory	Separate	I/O address space
Instructions	Normal	memory instructions	Special	I/O instructions
Programming		Easier	More specialized	
Latency		Low	Low,	but architecture-dependent
Suitable here		Yes	Less preferred	

Justification

Memory-mapped I/O is selected because it provides simple and fast access to sensor registers and integrates well with the processor's normal load/store operations. Vectored interrupts reduce interrupt-identification overhead. DMA can be added for bulk data transfer.

Conclusion

The proposed system achieves low interrupt latency and keeps **CPU utilization below 15%** by combining **memory-mapped I/O, vectored interrupts, interrupt prioritization, and DMA**.

2. I/O Subsystem for High-Speed NVMe Storage

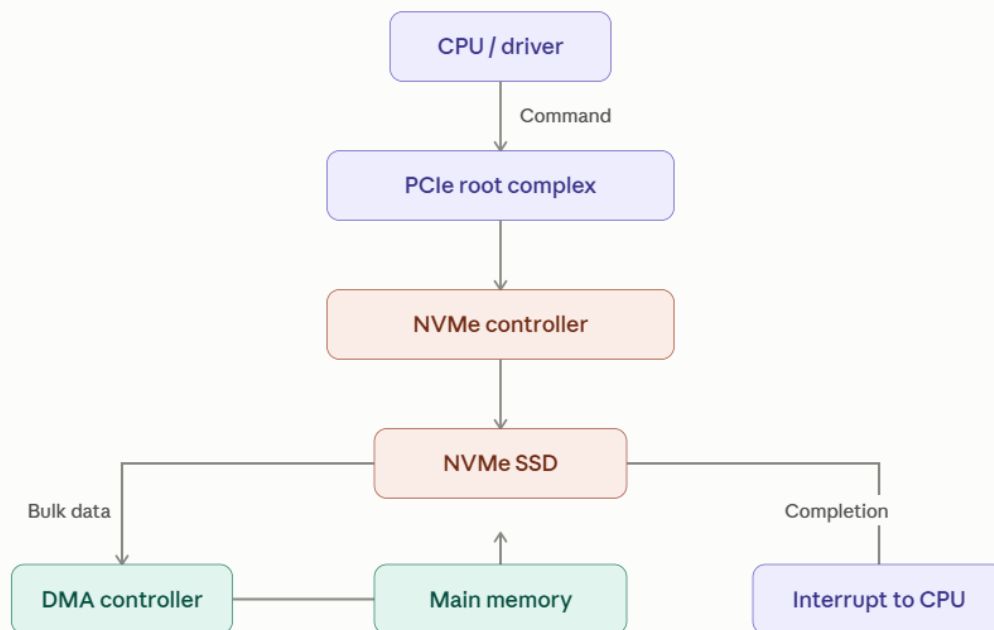
Constraints

- Throughput must be **greater than 3 GB/s**.
- Must use **NVMe over PCIe**.
- CPU should not handle bulk data transfer.
- DMA controller must be integrated with interrupt notification.

Solution

Use an **NVMe SSD connected through PCIe**, with **DMA-based data transfer** between the SSD and main memory.

Architecture



Working

1. CPU sends an NVMe command through the NVMe driver.
2. The NVMe controller processes the command.
3. Data is transferred directly between the NVMe device and RAM using **DMA**.
4. CPU does not copy the bulk data.
5. After completing the transfer, the NVMe controller generates an interrupt.
6. The CPU handles the completion notification.
7. NVMe queues allow multiple commands to be processed efficiently.

Why DMA?

Without DMA:

NVMe → CPU → Memory

The CPU would spend significant time moving data.

With DMA:

NVMe → DMA → Memory

The CPU only manages commands and completion notifications.

Throughput

PCIe provides high-bandwidth communication, while NVMe is designed for high-speed storage and parallel queues. The design should use an appropriate PCIe generation/lane configuration capable of sustaining >3 GB/s in practice.

Conclusion

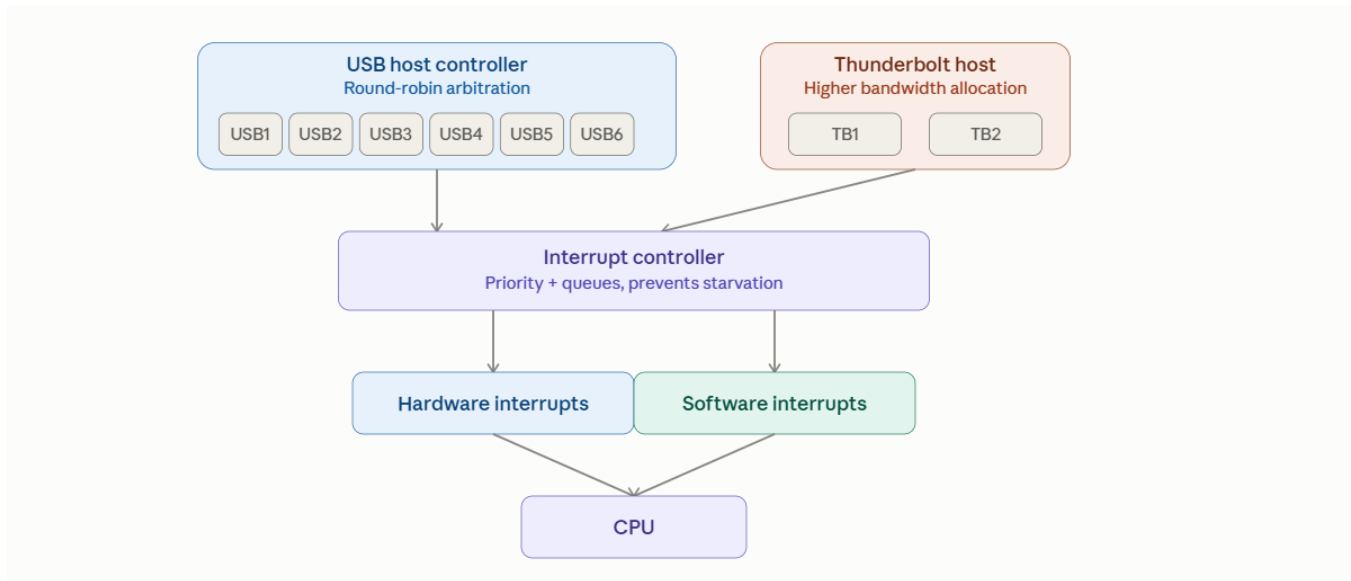
The optimal subsystem is **NVMe + PCIe + DMA + interrupt notification**. This achieves high throughput while keeping CPU involvement low.

3. Multi-Device Communication Architecture

Constraints

- **6 USB peripherals.**
- **2 high-speed Thunderbolt devices.**
- Avoid interrupt starvation.
- Use hardware and software interrupts.
- Ensure fair bus arbitration.

Architecture



Working

1. Six USB devices communicate through the USB host controller.
2. Two high-speed devices communicate through Thunderbolt.
3. Hardware interrupts are generated by devices requiring immediate attention.
4. Software interrupts are used for OS services and deferred/background processing.
5. An interrupt controller assigns priorities to interrupts.
6. High-priority interrupts are serviced first.
7. Lower-priority interrupts are placed in queues instead of being permanently blocked.
8. Fair bus arbitration allocates communication opportunities among devices.

Avoiding Interrupt Starvation

Use:

- Priority-based interrupt handling.
- **Round-robin service for devices of equal priority.**
- Interrupt masking only for very short critical sections.
- Interrupt queues.
- Deferred interrupt processing for lengthy operations.

Fair Bus Arbitration

A **round-robin arbitration mechanism** can be used:

USB1 → USB2 → USB3 → USB4 → USB5 → USB6

↑

↓

+----- Repeat -----+

Thunderbolt devices can receive higher bandwidth allocations when required, while arbitration prevents any device from monopolizing the shared resources.

Conclusion

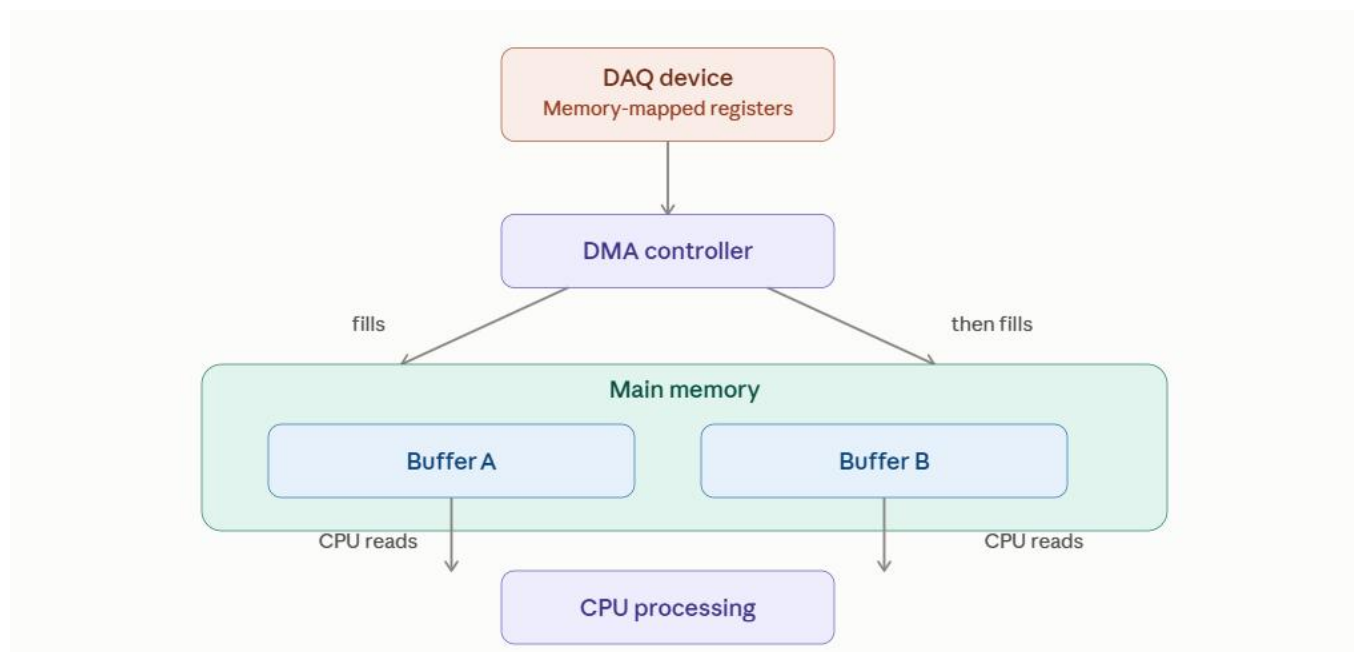
The architecture supports **6 USB + 2 Thunderbolt devices** while preventing interrupt starvation through **priorities, queues, round-robin arbitration, hardware interrupts, and software interrupts**.

4. Data Acquisition System – 500 MB/s

Constraints

- Continuous data stream of **500 MB/s**.
- Minimal CPU intervention.
- Compare **DMA transfer vs interrupt-driven I/O**.
- Use **memory-mapped I/O** for device registers.

Architecture



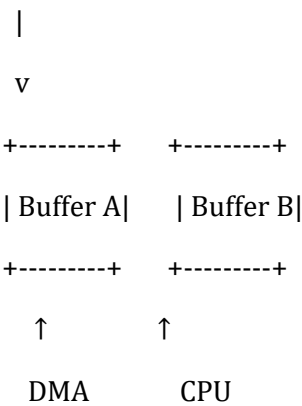
Working

1. The data acquisition device continuously generates data.
2. Its control/status registers are accessed using **memory-mapped I/O**.
3. DMA is configured with:
 - Source address
 - Destination address
 - Transfer size
4. The device transfers data directly to memory.

- DMA generates an interrupt after a block transfer is completed.
- CPU processes the completed buffer while DMA fills another buffer.

Double Buffering

Device



While DMA fills Buffer A, the CPU processes Buffer B. Then they switch.

DMA vs Interrupt-Driven I/O

Parameter	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s	Yes	No
Data movement	Direct device ↔ memory	Device ↔ CPU ↔ memory
Interrupt frequency	Block-based	Potentially very high
CPU overhead	Low	High
Performance	High	Lower

Why DMA is Better

At **500 MB/s**, interrupt-driven I/O would require frequent CPU intervention and could consume significant processing time. DMA transfers large blocks directly to memory and interrupts the CPU only when necessary.

Conclusion

The best solution is **memory-mapped I/O + DMA + double buffering**. It provides continuous 500 MB/s acquisition with minimal CPU intervention.

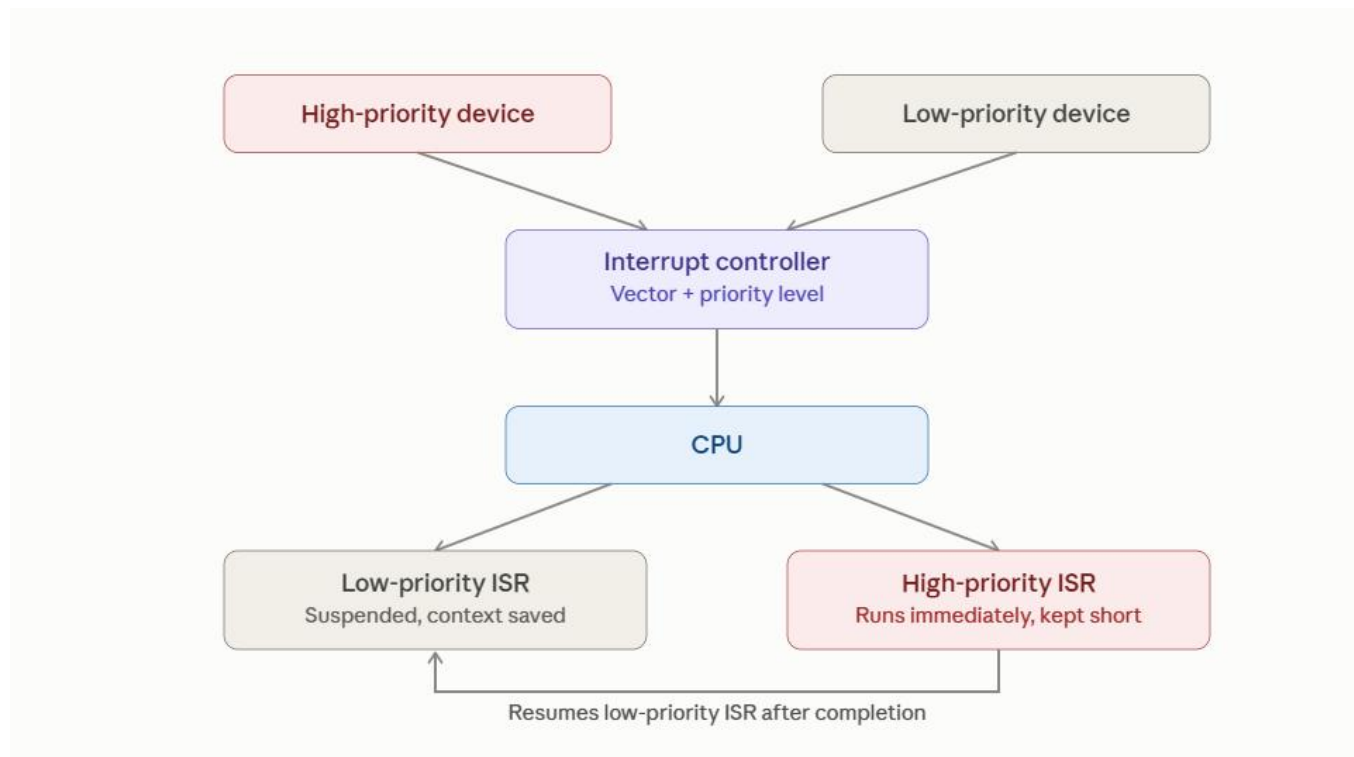
5. Interrupt Handling Mechanism

Constraints

- Support **nested interrupts**.
- Maximum interrupt latency < **10 μs**.
- Use vectored interrupts for high-priority devices.

- Use software interrupts for background tasks.

Architecture



Working

1. A device generates an interrupt.
2. The interrupt controller determines its priority.
3. A high-priority device receives a **vectored interrupt**.
4. CPU immediately jumps to its corresponding ISR.
5. If another higher-priority interrupt occurs during ISR execution, the current ISR can be temporarily suspended.
6. CPU saves the current context and services the higher-priority interrupt.
7. After completion, the previous ISR resumes.
8. Software interrupts are used for deferred/background processing.

Nested Interrupt Example

Main Program

|

v

Low Priority ISR

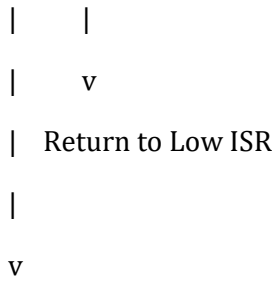
|

|---- High Priority Interrupt

| |

| v

| High Priority ISR



Main Program

Meeting the <10 μ s Requirement

To minimize interrupt latency:

- Use vectored interrupts.
- Keep ISRs short.
- Use a fast interrupt controller.
- Use priority levels.
- Avoid unnecessary interrupt masking.
- Move lengthy processing to deferred/software interrupt handlers.
- Use DMA for large data transfers.

Priority Structure

Priority 1 → Emergency / Real-Time Device

Priority 2 → High-Speed I/O

Priority 3 → Normal I/O

Priority 4 → Background Tasks

Conclusion

A **priority-based nested interrupt system with vectored interrupts** provides fast response for critical devices. Software interrupts handle non-critical background tasks, while short ISRs help maintain the required **<10 μ s maximum interrupt latency**.

Quick Final Summary

Question	Best Design
1	Memory-mapped I/O + vectored interrupts + DMA
2	NVMe + PCIe + DMA + completion interrupts
3	USB + Thunderbolt + priority interrupts + round-robin arbitration
4	Memory-mapped I/O + DMA + double buffering
5	Nested vectored interrupts + priority controller + software interrupts

